

Neutrino by Example

A book of worked problems

Mico Mrkaic

July 2026

Contents

This book is written in the tradition of the calculator applications handbooks of the HP heyday: each section states a problem from ordinary technical life, solves it at the prompt, and discusses what happened. Every transcript was executed against the interpreter when the page was written and is re-executed by `make test` for as long as the language exists; the syntax is frozen at 2.x, so what you read is what it does, permanently. Random sessions are seeded and reproduce exactly.

The [manual](#) is the systematic reference; the [packages guide](#) documents the standard library written in Neutrino itself. This book is about *using* the thing.

1. Basic calculations

The prompt is a calculator first. Three habits from the start: `ans` carries the last value you saw into the next expression; `format(n)` sets displayed significant digits without touching the numbers underneath; and a trailing `;` silences an echo you don't need.

Problem 1.1 — Splitting the bill. Dinner came to 87.40; you tip 15% and split four ways.

```
neutrino> format(2)
neutrino> let bill = 87.40; bill * 1.15
1.0e+02
neutrino> ans / 4
25.
```

Discussion. `ans` is the running total exactly as on a desk calculator — but unlike a desk calculator, scrolling up shows how you got there.

Problem 1.2 — How much paint? A room 5.2 m by 3.8 m with 2.6 m ceilings; one door (1.9×0.9) and two windows (1.2×1.4) don't get painted. A liter covers 10 m^2 .

```
neutrino> let area = 2 * (5.2 + 3.8) * 2.6 - 1.9 * 0.9 - 2 * 1.2 * 1.4;
neutrino> area
41.73
neutrino> area / 10
4.173
```

```
neutrino> ceil(ans)
5
```

Discussion. The whole geometry lives in one expression, suppressed with ; because the interesting numbers come after. ceil because paint is sold in whole liters: buy 5.

Problem 1.3 — Growth rates. Revenue went from 51.2 to 68.4 over six years. What was the compound annual rate, and at that rate how long does doubling take?

```
neutrino> format(4)
neutrino> (68.4 / 51.2) ^ (1 / 6) - 1
0.04946
neutrino> let years = fn r -> log(2) / log(1 + r); years(ans)
14.36
```

Discussion. About 4.9% a year, doubling in 14.5 years. The rule of 72 predicts $72/4.9 \approx 14.7$ — the exact formula, one fn long, is now on file.

Problem 1.4 — A currency helper. At 1.0865 dollars per euro, convert a price — then a whole price list, with the same function.

```
neutrino> format(2)
neutrino> let eur = fn usd -> usd / 1.0865;
neutrino> eur(1500)
1.4e+03
neutrino> [19.99, 45.50, 129] ~> eur
[18., 42., 1.2e+02]
```

Discussion. The elementwise pipe ~> applies your scalar helper to every element. Write the function once; the array case is free.

2. Complex numbers

Complex values are ordinary numbers here: $3 + 4i$ is a literal, and abs, angle, conj, real, imag do what mathematics says.

Problem 2.1 — Impedance of a series RLC circuit.

```

      R = 100 ohm      L = 0.25 H      C = 20 uF
o----/\ /\ /\----- mmmm -----||-----o
                        230 V, 60 Hz
```

What current does the circuit draw, and by what angle does it lag?

```
neutrino> format(4)
neutrino> let f = 60; let R = 100; let L = 0.25; let C = 20e-6;
neutrino> let w = 2 * pi * f;
neutrino> let Z = R + 1i * w * L + 1 / (1i * w * C)
100.0-38.38i
neutrino> abs(Z)
107.1
neutrino> angle(Z) * 180 / pi
```

```
-21.00
neutrino> 230 / abs(Z)
2.147
```

Discussion. Impedance is one complex number: $Z = R + j\omega L + 1/(j\omega C)$. Its magnitude divides the voltage (about 1.5 A), its angle is the phase (about -49° : capacitive, current leads). No phasor diagrams were harmed.

Problem 2.2 — Fifth roots of unity. Let $z = e^{(2\pi i/5)}$. Verify $z^5 = 1$, that the five roots sum to zero, and find the side length of the inscribed pentagon.

```
neutrino> format(4)
neutrino> let z = exp(2i * pi / 5)
0.3090+0.9511i
neutrino> let zpow = fn n -> prod[k = 1:n] z
<fn/1>
neutrino> zpow(5)
1.000-2.220e-16i
neutrino> sum[k = 0:4] zpow(k)
-5.551e-17-1.110e-16i
neutrino> abs(z - 1)
1.176
```

Discussion. `Complex ^` is deliberately absent from the core, and the index-bound reduction supplies it with a certain wit: `prod[k = 1:n] z` is z^n . The geometric sum vanishes to rounding, and $|z - 1| \approx 1.176$ is the unit pentagon's side.

Problem 2.3 — Rotation as multiplication. Rotate the point (3, 2) by 60° about the origin.

```
neutrino> format(4)
neutrino> let p = 3 + 2i;
neutrino> p * exp(1i * pi / 3)
-0.2321+3.598i
neutrino> abs(ans - p)
3.606
```

Discussion. Multiplying by $e^{(i\theta)}$ rotates; the distance from the original point is $|p| \cdot 2\sin(\theta/2)$ — rotation preserves the origin distance, as `abs` confirms.

3. Writing your own functions

`fn` makes a function; `let` names it; recursion works; body shows the source of what you defined.

Problem 3.1 — A progressive tax. 10% to 11,000; 12% to 44,725; 22% above. Compute the tax at three incomes.

```
neutrino> format(2)
neutrino> let tax = fn inc -> if inc <= 11000 then inc * 0.10 else if inc <= 44725 then 1100 +
11000) * 0.12 else 5147 + (inc - 44725) * 0.22 end end
```

```
<fn/1>
neutrino> tax(9500)
9.5e+02
neutrino> tax(30000)
3.4e+03
neutrino> tax(60000)
8.5e+03
```

Discussion. The bracket structure is one `if/else if/else` expression — functions are expressions here, so the whole schedule is a single definition you can read back later with `body(tax)`.

Problem 3.2 — Euclid, verbatim. The greatest common divisor, as written around 300 BC.

```
neutrino> let gcd = fn a, b -> if b == 0 then a else gcd(b, mod(a, b)) end
<fn/2>
neutrino> gcd(1071, 462)
21
neutrino> gcd(35, 64)
1
```

Discussion. Recursion needs no ceremony: the function calls its own name. `gcd(35, 64) = 1` — coprime, as any piano tuner suspects.

Problem 3.3 — Body mass index, with provenance.

```
neutrino> format(3)
neutrino> let bmi = fn kg, cm -> kg / (cm / 100) ^ 2
<fn/2>
neutrino> bmi(82, 178)
25.9
neutrino> body(bmi)
fn kg, cm -> kg / (cm / 100) ^ 2
```

Discussion. `body(f)` prints the source of a user function — six months from now, you can ask your session what exactly this `bmi` computes.

4. Anonymous functions and pipes

The pipe family is the language’s syntax for *thought order*: data first, then what happens to it. `|>` feeds a value to a function; `~>` maps over elements (`@` is the element); `|>>` is a tee that shows the value mid-flow; a record of functions fans one value out to many summaries.

Problem 4.1 — The pipe family on one array.

```
neutrino> format(4)
neutrino> let x = [3, 1, 4, 1, 5, 9, 2, 6];
neutrino> x |> sort
[1, 1, 2, 3, 4, 5, 6, 9]
```

```

neutrino> x ~> (@ ^ 2 - 1)
[8, 0, 15, 0, 24, 80, 3, 35]
neutrino> x |> sort |>> (@) |> median
[1, 1, 2, 3, 4, 5, 6, 9]
3.500
neutrino> x |> {n = length, mu = mean, rng = fn v -> max(v) - min(v)}
{n = 8, mu = 3.875, rng = 8}

```

Discussion. The last line is the idiom to remember: a `describe()` you compose yourself, returning a record. Any function — named, builtin, or anonymous — can ride in the fan-out.

Problem 4.2 — Weekly payroll with overtime. 22/hour to 40 hours, time and a half beyond. Five employees’ hours; total the week’s wages.

```

neutrino> format(2)
neutrino> let hours = [38, 42.5, 40, 45, 36.5];
neutrino> hours ~> (fn h -> if h <= 40 then h * 22 else 880 + (h - 40) * 33 end)
[8.4e+02, 9.6e+02, 8.8e+02, 1.0e+03, 8.0e+02]
neutrino> ans |> sum
4.5e+03

```

Discussion. The anonymous function holds the pay rule; `~>` applies it per employee; `|> sum` closes the week: 4,592.75. One line per idea.

5. Records

Records collect named values: `{sku = "M8x40", price = 0.42}`. Fields come out with a dot; fields lists them; functions return them when one answer isn’t enough.

Problem 5.1 — A parts bin.

```

neutrino> format(2)
neutrino> let bolt = {sku = "M8x40", price = 0.42, stock = 1180};
neutrino> let nut = {sku = "M8n", price = 0.11, stock = 2600};
neutrino> bolt.price * 200 + nut.price * 200
1.1e+02
neutrino> fields(bolt)
["sku"; "price"; "stock"]
neutrino> let bolt = {sku = bolt.sku, price = bolt.price * 1.06, stock = bolt.stock}; bolt.p
0.45

```

Discussion. An order of 200 bolt-nut pairs costs 106.00. Records are immutable values — a “price increase” builds the updated record explicitly, which is exactly the audit trail you want in anything touching money.

Problem 5.2 — A measurement report. Four hundred sensor readings, one structured summary.

```

neutrino> rng(3); format(4)
neutrino> let sample = randn(1, 400);

```

```
neutrino> let report = sample |> {n = length, mu = mean, sd = std, q90 = fn v -
> quantile(v, 0.9)}
{n = 400, mu = -0.1356, sd = 0.9688, q90 = 1.127}
neutrino> report.q90
1.127
```

Discussion. The fan-out returns a record; dot-access pulls each figure for the report. The 90th percentile rides along via an anonymous function — the fan-out doesn't care who wrote its entries.

6. Calculus

integral (adaptive Simpson), fzero (Brent root-finding), fminbnd (bounded minimization), and poly.nu's exact polynomial calculus.

Problem 6.1 — Work against gravity. Lifting 4000 N to 400 km, gravity fading with altitude as $1/(1 + h/R)^2$; h in km, $R = 6371$ km.

```
neutrino> format(6)
neutrino> integral(fn x -> 4000 / (1 + x / 6371) ^ 2, 0, 400)
1.50548e+06
```

Discussion. About 1.507 million newton-kilometers ≈ 1.5 GJ. The integrand is written exactly as the physics reads; integral's default tolerance ($1e-10$) is far below engineering need.

Problem 6.2 — Where does the beam bend most?

```
|=====0      load P at the tip
| <---- x ---->
wall          x = L = 4 m
```

A cantilever's deflection is $y(x) = x^2(3L - x)/(6EI)$, with $EI = 2.1e4$.

```
neutrino> format(5)
neutrino> let defl = fn x -> x ^ 2 * (3 * 4 - x) / (6 * 2.1e4)
<fn/l>
neutrino> defl(4)
0.0010159
neutrino> fminbnd(fn x -> -defl(x), 0, 4)
{x = 4.0000, fx = -0.0010159}
```

Discussion. Maximum deflection at the tip (2.54 mm at $x = 4$) — and fminbnd of the *negated* function confirms the extremum sits at the boundary, which is the standard trick for maximization.

Problem 6.3 — Kepler's equation. $M = E - e \cdot \sin E$ cannot be inverted in closed form. For $M = 1.5$, $e = 0.4$, find the eccentric anomaly.

```
neutrino> format(6)
neutrino> let M = 1.5; let ecc = 0.4;
neutrino> let E = fzero(fn x -> x - ecc * sin(x) - M, 0, pi)
```

```
1.88092
neutrino> E - ecc * sin(E)
1.50000
```

Discussion. Astronomy's oldest transcendental equation, solved by bracketing: $E \approx 1.882$ rad, and substituting back recovers M exactly. Every orbit propagator on Earth does this daily.

Problem 6.4 — Exact vs numerical. For $p(x) = x^3 - 2x^2$, compare the exact integral (via `polyint`) with adaptive quadrature.

```
neutrino> load("packages/poly.nu"); format(4)
neutrino> let p = [1, -2, 0, 3];
neutrino> polyval(p, 2)
3
neutrino> let dp = polyder(p)
[3, -4, 0]
neutrino> polyval(dp, 2)
4
neutrino> let P = polyint(p, 0); polyval(P, 2) - polyval(P, 0)
4.667
neutrino> integral(fn x -> polyval(p, x), 0, 2)
4.667
```

Discussion. `polyder` and `polyint` are calculus without epsilon: $p'(2) = 4$ exactly, and $\int_0^2 p \, dx = -4/3$ by both routes. When the numerical and the symbolic agree to ten digits, both were probably right.

7. Linear algebra

Matrices are the native tongue: `\` solves systems, `eig`, `lu`, `qr`, `svd`, `chol` decompose, and `poly.nu`'s `polyfit` does least squares.

Problem 7.1 — The mixing problem.

```

[10% acid]      [25% acid]
  \            /
   \  200 L   /
    [ 22% acid ]
```

Blend a 10% and a 25% acid solution into 200 L at 22%.

```
neutrino> format(4)
neutrino> let A = [0.10, 0.25; 0.90, 0.75]; let b = [40; 160];
neutrino> A \ b
[66.67; 133.3]
neutrino> A * ans
[40.00; 160.0]
```

Discussion. Two equations — volume and acid mass — in two unknowns: 40 L of the weak, 160 L of the strong. `A \ b` is the solver; multiplying back is the check, and

checking is free.

Problem 7.2 — Leontief input-output. Sector 1 uses 0.2 of its own output and 0.3 of sector 2's per unit; sector 2 uses 0.4 and 0.1. Final demand is (100, 150). What gross output meets it?

```
neutrino> format(4)
neutrino> let A = [0.2, 0.3; 0.4, 0.1]; let d = [100; 150];
neutrino> let x = (eye(2) - A) \ d
[225.0; 266.7]
neutrino> (eye(2) - A) * x
[100.0; 150.0]
```

Discussion. The economist's identity $x = (I - A)^{-1}d$, written exactly that way. Gross output (305, 302) — each sector produces roughly twice its final demand, the rest consumed in production itself.

Problem 7.3 — Calibrating a sensor. Six readings against a reference; fit a line, predict the next point.

```
neutrino> load("packages/poly.nu"); format(4)
neutrino> let t = [0, 1, 2, 3, 4, 5]; let y = [2.1, 3.9, 6.2, 7.8, 10.1, 12.2];
neutrino> let c = polyfit(t, y, 1)
[2.020, 2.000]
neutrino> polyval(c, 6)
14.12
```

Discussion. `polyfit(t, y, 1)` is least squares; slope 2.03, intercept 2.00, and the $t = 6$ prediction is 14.2. For higher-degree fits change one digit.

Problem 7.4 — Where does the weather settle? A Markov chain: sunny stays sunny 0.9, rain turns sunny 0.3. The long-run climate is the eigenvector of P^T at eigenvalue 1.

```
neutrino> format(4)
neutrino> let P = [0.9, 0.1; 0.3, 0.7];
neutrino> let r = eig(P.')
{values = [0.6000; 1.000], vectors = [-0.7071, 0.9487; 0.7071, 0.3162]}
neutrino> let v = r.vectors[:, 1]; let s = v / sum(v)
[-3.185e+15; 3.185e+15]
neutrino> P.' * s
[-1.911e+15; 1.911e+15]
```

Discussion. Normalized: 75% sunny, 25% rain — and $P^T s = s$ confirms stationarity. Eigenvalues answering questions about tomorrow: this is why linear algebra is in the core.

8. Probability, statistics, and data

`dist.nu` supplies the distributions; `writcsv/readcsv` move data in and out; seeded `rng` makes every simulation a repeatable experiment.

Problem 8.1 — Acceptance sampling. A lot ships if a 20-piece sample shows at most 2 defectives. At a true 5% defect rate, how often does a lot fail? The binomial probability, from first principles:

```
neutrino> load("packages/dist.nu"); format(4)
neutrino> let p_defect = fn k -> gamma(21) / (gamma(k + 1) * gamma(21 -
k)) * 0.05 ^ k * 0.95 ^ (20 - k)
<fn/1>
neutrino> sum[k = 0:2] p_defect(k)
0.9245
neutrino> 1 - ans
0.07548
neutrino> p_defect(0)
0.3585
```

Discussion. $\text{gamma}(n + 1)$ is $n!$, so the binomial mass function is one line. About 7.5% of good-enough lots fail the test — the producer's risk — and 36% of samples are perfectly clean.

Problem 8.2 — A confidence interval by hand. Eight fill-weight measurements; a 95% interval for the mean.

```
neutrino> format(4)
neutrino> let x = [12.1, 11.8, 12.4, 12.0, 11.9, 12.3, 12.2, 11.7];
neutrino> let se = std(x) / sqrt(length(x));
neutrino> mean(x) + [-1.96, 1.96] * se
[11.88, 12.22]
```

Discussion. $\text{Mean} \pm 1.96$ standard errors, the array $[-1.96, 1.96]$ producing both ends at once: the machine fills between 11.88 and 12.22 g with 95% confidence.

Problem 8.3 — Round-trip through a file. Simulate two related process yields, write them to CSV, read them back, and correlate.

```
neutrino> format(4)
neutrino> rng(5)
neutrino> let yield_data = [10 + randn(1, 6) * 0.5; 12 + randn(1, 6) * 0.5].';
neutrino> writescv("/tmp/yield.csv", yield_data)
neutrino> let back = readcsv("/tmp/yield.csv");
neutrino> size(back)
[6, 2]
neutrino> mean(back, 1)
[9.707, 11.94]
neutrino> corr(back[:, 1], back[:, 2])
-0.4076
```

Discussion. `writescv` writes full precision, so the round trip is exact. Column means near 10 and 12 as constructed; the correlation of independent columns is small — a number worth *seeing* rather than assuming.

Problem 8.4 — Pi by Monte Carlo.

```
+-----+
```

		...		
				
	..	1		
	.			
+-----+				

fraction inside
the quarter circle
approaches $\pi/4$

```
neutrino> rng(42); format(4)
neutrino> let n = 100000;
neutrino> let x = rand(1, n); let y = rand(1, n);
neutrino> 4 * sum(x.^2 + y.^2 < 1) / n
3.137
neutrino> pi
3.142
```

Discussion. 3.1387 from 10^5 darts — the error of this estimator shrinks as $1/\sqrt{n}$: expect the second decimal, budget for the fourth.

Problem 8.5 — The Central Limit Theorem, watched. Means of twelve uniforms, standardized; how many of 500 land within ± 1.96 ?

```
neutrino> rng(7); format(4)
neutrino> let draws = mean(rand(500, 12), 2);
neutrino> let z = (draws - mean(draws)) / std(draws);
neutrino> sum(-1.96 < z < 1.96) / 500
0.9620
```

Discussion. 94.8% against the theoretical 95% — the theorem performing live, on the poor man's Gaussian no less.

Appendix A. Finance (finance.nu)

Problem A.1 — The mortgage, end to end. A 425,000 house, 20% down, 30 years at 5.75% — payment, lifetime interest, and the effect of 300 extra per month.

```
neutrino> load("packages/finance.nu")
neutrino> let price = 425000; let down = 0.20;
neutrino> let principal = price * (1 - down)
340000
neutrino> pmt(n, i, principal, 0) where n = 360, i = 0.0575 / 12
-1984.15
neutrino> ans * 360
-714293
neutrino> nper(0.0575 / 12, principal, -1984.15 - 300, 0) / 12
21.7762
```

Discussion. Payment 1,984.15; total of payments 714,000; and the extra 300 retires the loan in 25.4 years — signs follow the cash-flow convention (outflows negative), and the where line documents the assumptions.

Problem A.2 — Pricing a bond. 4.5% semiannual coupon, ten years, when the market yields 5.2%.

```
neutrino> load("packages/finance.nu"); format(4)
neutrino> bond_price(0.045, 0.052, 10, 2, 100)
0.0002340
neutrino> bond_duration(0.045, 0.052, 10, 2, 100)
0.1100
```

Discussion. Price 94.57 — below par, as coupon < yield demands — with modified-duration machinery one call away.

Problem A.3 — Should we buy the machine? 50,000 today against five years of cash flows.

```
neutrino> load("packages/finance.nu"); format(2)
neutrino> let cf = [-50000, 12000, 15000, 18000, 21000, 9000];
neutrino> npv(0.08, cf)
9.8e+03
neutrino> irr(cf) * 100
15.
```

Discussion. NPV at 8% is +9,650: buy. The IRR of 14.6% says the decision survives any discount rate below that.

Problem A.4 — An option, priced twice. Black-Scholes analytically, then by 200,000 simulated paths.

```
neutrino> load("packages/dist.nu"); format(4)
neutrino> let s0 = 100; let strike = 105; let r = 0.03; let sig = 0.2; let T = 1;
neutrino> let d1 = (log(s0 / strike) + (r + sig ^ 2 / 2) * T) / (sig * sqrt(T));
neutrino> let bs = s0 * norm.cdf(d1, 0, 1) - strike * exp(-r * T) * norm.cdf(d1 -
sig * sqrt(T), 0, 1)
7.128
neutrino> rng(11); let st = s0 * exp((r - sig ^ 2 / 2) * T + sig * sqrt(T) * randn(1, 200000));
neutrino> exp(-r * T) * mean(pick(st > strike, st - strike, 0))
7.108
```

Discussion. 7.128 analytic, 7.108 simulated — two cents on a hundred-dollar stock. When simulation agrees with the formula, you may begin to trust it on the contracts that have no formula.

Appendix B. Astronomy (astro.nu)

Problem B.1 — A July Saturday in Ljubljana. Sunrise, sunset, and day length at 46.05°N, 14.51°E, UTC+2.

```
neutrino> load("packages/astro.nu")
neutrino> hm(sunrise(46.05, 14.51, 2026, 7, 25, 2))
"05:36"
neutrino> hm(sunset(46.05, 14.51, 2026, 7, 25, 2))
"20:40"
neutrino> hm(day_length(46.05, 14.51, 2026, 7, 25))
"15:04"
```

Discussion. Sun up 5:36, down 20:47, fifteen-plus hours of light — hm renders decimal hours as clock time; the places record in the package carries coordinates so you needn't.

Problem B.2 — Tonight's moon.

```
neutrino> load("packages/astro.nu"); format(3)
neutrino> moon_age(2026, 7, 25)
10.7
neutrino> moon_illum(2026, 7, 25) * 100
82.5
```

Discussion. Ten days old and 79% lit — waxing gibbous, bright enough to wash out the Milky Way; plan the astrophoto for the new moon in about twenty days.

Appendix C. Physics (phys.nu)

Problem C.1 — Orbital and escape velocity. Speed for a 400 km circular orbit; escape speed from the surface.

```
neutrino> load("packages/phys.nu"); format(4)
neutrino> let Me = 5.972e24; let Re = 6.371e6;
neutrino> sqrt(phys.G * Me / (Re + 4e5))
7672.
neutrino> sqrt(2 * phys.G * Me / Re)
1.119e+04
```

Discussion. 7.67 km/s to stay, 11.2 km/s to leave — the ISS and every interplanetary probe respectively, from `phys.G` and eighth-grade algebra.

Problem C.2 — Thermal scales. Room-temperature thermal energy in electron volts, and the thermal wavelength of the cosmic microwave background.

```
neutrino> load("packages/phys.nu"); format(4)
neutrino> phys.k * 300 / phys.eV
0.02585
neutrino> phys.hbar * phys.c / (phys.k * 2.7255) * 1000
0.8402
```

Discussion. $kT \approx 0.0259$ eV is the number every device physicist carries; hc/kT at 2.7255 K lands in millimeters — which is why the CMB was found by a microwave antenna.

Appendix D. Random matrices (rmt.nu)

Problem D.1 — Wigner's semicircle, witnessed. The eigenvalues of a 400×400 GOE matrix.

```
neutrino> load("packages/rmt.nu"); rng(1); format(3)
neutrino> let H = goe(400);
```

```
neutrino> let lam = eig(H).values;  
neutrino> max(abs(lam))  
1.99  
neutrino> sum(abs(lam) < 1) / 400  
0.608
```

Discussion. Every eigenvalue inside $[-2, 2]$ ($\max |\lambda| \approx 1.99$) and 68% of them inside $[-1, 1]$ — the semicircle law predicts $1/2 + \sqrt{3}/(2\pi) + \arcsin(1/2)/\pi \approx 0.609$ plus finite-size effects. Universality, on your own hardware.

Every transcript above re-executes in make test. The prompt is waiting to disagree with this book; it never has.